

programación funcional

en Scala

principios de la programación funcional

la programación funcional es **declarativa**

una componente es declarativa si es **independiente del resto de la computación** (sólo se comunica por parámetros - retorno de función), es decir, no usa variables fuera de su alcance (variables globales)

implicaciones de la independencia declarativa

una componente declarativa...

es **determinística**: toda ejecución con los mismos valores de parámetros tendrá el mismo resultado

→ va a tener **transparencia referencial**

→ vamos a poder usar las funciones como valores (**alto nivel**)

no tiene efectos secundarios, es decir, no tiene efectos fuera del alcance de la función

no se maneja el **estado** de forma explícita

por qué Scala

lenguaje multi-paradigma

una sintaxis moderna, concisa y elegante (menos verboso que Java)

sintaxis que facilita patrones útiles de programación

con seguridad de tipos (tipado estático)

se ejecuta sobre la JVM, interoperabilidad completa con Java

- se puede usar como sustituto de Java, o mezclado con Java
- puede usar las librerías de Java
- puede usar todas las herramientas del ecosistema Java (Ant, Maven, JUnit)

programación funcional en Scala

basado en materiales de

<https://www.tpunder.com/blog/2025-03-26-a-brief-intro-to-scala/>

sin efectos secundarios: variables mutables y valores inmutables

```
// variable
```

```
var foo = "foo" // funciona porque foo es una var
```

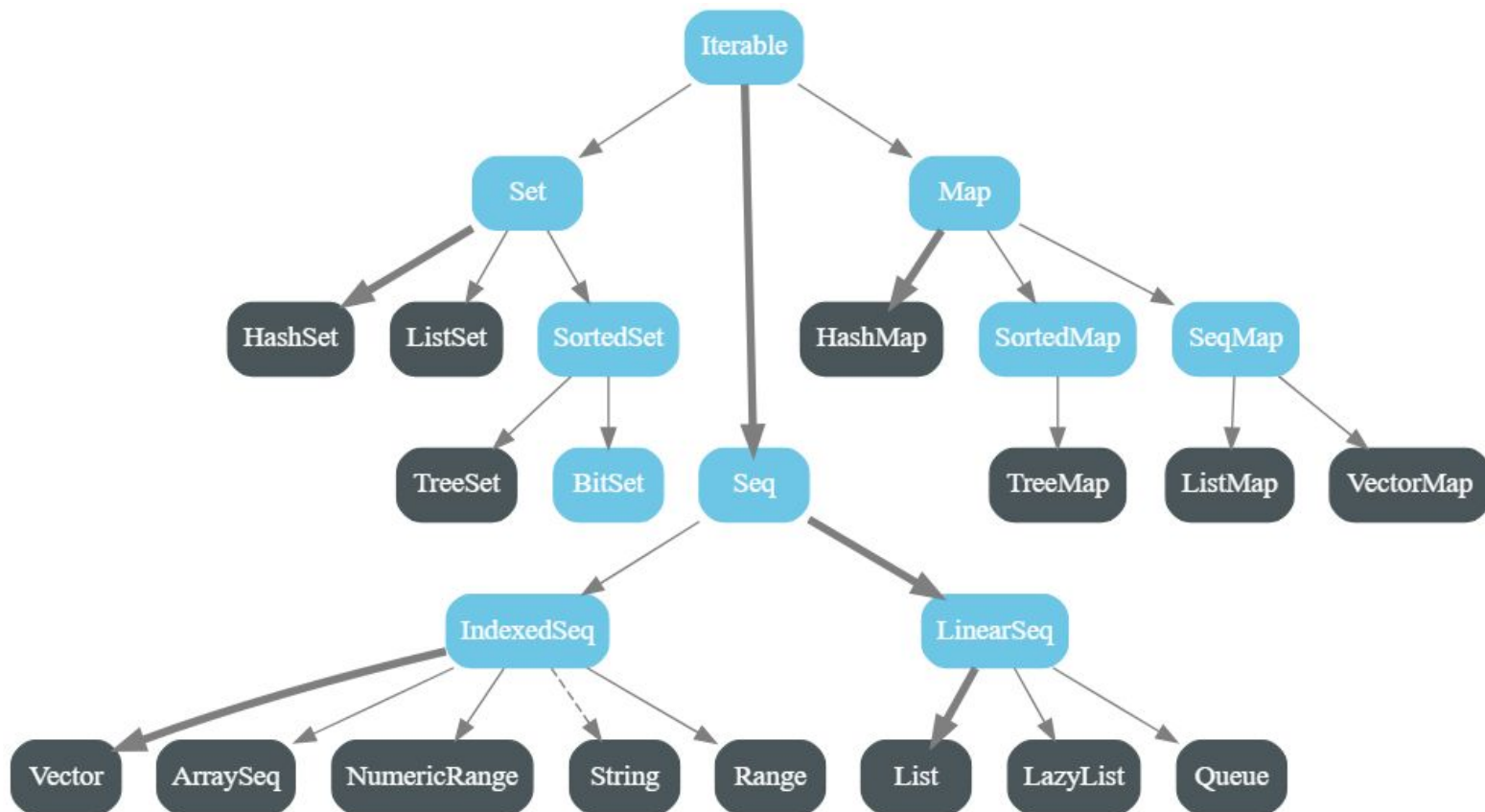
```
foo = "bar"
```

```
// valor
```

```
val bar = "bar" // no compila porque bar es un val
```

```
bar = "foo"
```

scala.collection.immutable



funciones de primera clase

```
// crear función anónima
```

```
val plusOne = (x: Int) => x + 1
```

```
plusOne(5)
```

```
// plusFoo can reference any values/variables in scope
```

```
var foo = 1
```

```
val plusFoo = (x: Int) => x + foo
```

```
plusFoo(5) // 6
```

```
foo = 5
```

```
plusFoo(5) // 10
```

no es funcional

alto orden: tomar funciones como parámetro

```
def ejecutar(f: Int => Int) = f(1)
```

```
ejecutar(plusOne)      // 2
```

```
ejecutar(x => x + 1)   // 2
```

```
ejecutar(_ + 1)       // 2
```

propiedades valiosas de la programación declarativa

alto nivel

```
// Check if string has uppercase character

boolean hasUpperCase = false;

for(int i = 0; i < name.length(); i++) {

    if(Character.isUpperCase(name.charAt(i))) {

        hasUpperCase = true;

        break;

    }

}

// Check if string has uppercase character

val hasUpperCase = name.exists(_.isUpperCase)
```

otras propiedades valiosas: pattern matching

```
def what(any: Any) = any match {  
  case i: Int      => "It's an Int"  
  case s: String => "It's a String"  
  case _          => "I don't know what it is"  
}
```

```
what(123)      // "It's an Int"
```

```
what("hello")  // "It's a String"
```

```
what(false)    // "I don't know what it is"
```

otras propiedades valiosas: pattern matching

```
val nums = List(1,2,3)
```

```
// Pattern matching to create 3 vals
```

```
val List(a,b,c) = nums
```

```
a    // 1
```

```
b    // 2
```

```
c    // 3
```

otras propiedades valiosas: inferencia de tipos

```
val sum = 1 + 2 + 3 // Int
val lst = List(1, 2, 3) // List[Int]
val map = Map("abc" -> List(1, 2, 3)) // Map[String, List[Int]]
```

ejemplos imperativo vs. funcional

sumar una lista

```
def sumList(nums: List[Int]): Int = {  
  var total = 0  
  
  for (n <- nums) {  
    total += n  
  
  }  
  
  total  
}
```

```
def sumList(nums: List[Int]): Int =  
  nums.foldLeft(0)(_ + _)
```

filtrar números impares

```
def evenNumbers(nums: List[Int]): List[Int] = {  
  var result = List[Int]()  
  for (n <- nums) {  
    if (n % 2 == 0) {  
      result = result :+ n  
    }  
  }  
  result  
}
```

```
def evenNumbers(nums: List[Int]): List[Int] =  
  nums.filter(_ % 2 == 0)
```

subir al cuadrado los elementos de una lista

```
def squareNumbers(nums: List[Int]): List[Int] = {  
  var result = List[Int]()  
  
  for (n <- nums) {  
    result = result :+ (n * n)  
  }  
  
  result  
}
```

```
def squareNumbers(nums: List[Int]): List[Int] =  
  nums.map(n => n * n)
```

factorial

```
def factorial(n: Int): Int = {  
  var result = 1  
  for (i <- 1 to n) {  
    result *= i  
  }  
  result  
}
```

```
import scala.annotation.tailrec  
  
def factorial(n: Int): Int = {  
  @tailrec  
  def loop(n: Int, acc: Int): Int =  
    if (n <= 1) acc  
    else loop(n - 1, n * acc)  
  
  loop(n, 1)  
}
```

agrupar palabras por su longitud (para wordle ;-)

```
def groupByLength(words: List[String]): Map[Int, List[String]] = {  
  var result = Map[Int, List[String]]()  
  
  for (w <- words) {  
    val len = w.length  
  
    val current = result.getOrElse(len, List())  
  
    result = result.updated(len, current :+ w)  
  }  
  
  result  
}
```

```
def groupByLength(words: List[String]): Map[Int,  
List[String]] =  
  
  words.groupBy(_.length)
```

división segura (sin que lance un error)

```
def safeDivide(a: Int, b: Int): Option[Int] = {  
  if (b == 0) {  
    None  
  } else {  
    Some(a / b)  
  }  
}
```

```
def safeDivide(a: Int, b: Int): Option[Int] =  
  Option(b)  
    .filter(_ != 0)  
    .map(_ => a / b)
```

contar frecuencia de palabras

```
def wordCount(words: List[String]): Map[String, Int] = {
```

```
  var counts = Map[String, Int]()
```

```
  for (w <- words) {
```

```
    val count = counts.getOrElse(w, 0) + 1
```

```
    counts = counts.updated(w, count)
```

```
  }
```

```
  counts
```

```
}
```

materiales de referencia

<https://docs.scala-lang.org/overviews/scala-book/functional-programming.html>